CS 4590 - Project Deliverable Two

By: Connor Case

Spring 2026

1 Recap of Project Deliverable One

In PD1, I introduced the core idea of a context-aware audio notification system designed to reduce the need for visual attention when receiving notifications. The goal was to allow users to understand both the type and importance of incoming notifications purely through sound. I focused on three main principles: making sounds learnable, adapting them to different contexts, and prioritizing important information without overwhelming the user.

Using the defined four contexts (working out, walking, socializing, and presenting), I described how each one affects notification delivery. These contexts influence factors like volume, level of detail, and whether certain notifications should be suppressed entirely. The key idea was that the same notification should sound and behave differently depending on the user's environment and cognitive load.

Finally, I designed sonification strategies for five event types: Twitter, email, text messages, phone calls, and voicemail. Each type uses a mix of auditory icons, earcons, and text-to-speech, along with parameters like pitch, rhythm, and timbre to encode additional information such as priority or sentiment.

Context - Technical Details

The technical details for the contexts are summarized below. Note that the details pertain to all notification types during each context.

1. **Working Out**: Higher volume to overcome ambient noise, allows richer audio (earcons + text-to-speech), minimal suppression of notifications due to high interruptibility.
2. **Walking**: Low–medium volume to preserve environmental awareness, shorter and less intrusive sounds, all priorities allowed but compressed in duration.
3. **Coffee Shop**: Lower volume with minimal, subtle earcons, reduced use of text-to-speech except for high-priority notifications, designed to avoid disrupting conversation.
4. **Presentation**: Most notifications suppressed except highest priority, concise sounds at medium volume, no unnecessary layering or extended audio.

Notification Type - Technical Details

The technical details for the notification types are summarized below:

1. **Twitter**: Short chirp-like auditory icon with pitch representing priority and brightness reflecting engagement; text-to-speech only for highest priority and if it has more than 10,000 favorites.
2. **Email**: Earcon resembling an "envelope opening," with duration and volume tied to priority; chord quality reflects sentiment; text-to-speech for higher priority.
3. **Text Message**: Simple tap/pop auditory icon with repetition indicating priority; consistent use of short text-to-speech for message content.
4. **Phone Call**: Classic ringing auditory icon with repetition rate and prominence scaled by priority; designed to stand out clearly.
5. **Voice Mail**: Low, warm earcon followed by text-to-speech summary; pitch and subtle filtering encode priority and sentiment.

2 Architecture of the Sonification Simulator

The way that I designed this sonification system is two-fold. First, I have generic settings that apply to all notifications for each context. These are called the "context parameters". These have generic settings like text-to-speech speed and volume. Once I've loaded and started the stream from the JSON file, I pass a handler that will handle each notification.

At a high level, the system is built around a central event pipeline. The simulator loads a JSON data stream and schedules notifications using a notifier module. Each event is then passed into a single handler function (`handleNotification`), which acts as the entry point for all sonification. This handler first checks whether the event type is currently enabled through user-controlled filters, and then applies the appropriate context parameters before passing the event into a dedicated sonification function.

From there, the system branches into specialized functions for each notification type (Twitter, Email, Text Message, Phone Call, Voice Mail). Each of these functions encapsulates its own sound design logic, including oscillator usage, audio buffers, filters, and optional text-to-speech. This modular structure makes it easy to tweak or extend individual notification types without affecting the rest of the system. Most importantly I used conditionals in each specialized notification function that checks for what the current context is and will do text-to-speech in high interuptability environments, or only give soft sounds in low interuptability contexts. Additionally, I maintain a list of active audio nodes so that I can cleanly stop and reset all sounds when switching contexts or restarting the simulator.

Finally, I made a few implementation choices to improve usability. For example, I introduce a slight delay before triggering sonification to prevent overlaps, and I scale volume based on priority so that more important notifications stand out. I also selectively enable text-to-speech depending on both priority and context (like suppressing detailed speech during presentations). Lastly, I implemented a priority-queue that will selectively handle the higher priority notifications first. This prevents bombarding the user with a ton of earcons and auditory icons.

3 Using the Simulator

To experience the different scenarios, the user can dynamically switch between contexts, data streams, and notification filters through the UI controls. Changing the context immediately updates both the background audio and the parameters used for future notifications. The play/stop and reset controls are intuitive and make it simple to restart and compare different configurations.

Additionally, for both debugging and for grading convienience, I've made it so that there's a status log on the right side of the radio buttons. This let's you see what's going on "beneath the hood", and allowed me to understand how my priority queue and other notifications were being handled.